

Relatório de Problemas de Coloração de Arestas

Rafael Lazaro Costa, Augusto Benedito, João Pedro Costa, Victor Souza

March 17, 2025

1 Introdução

O problema de coloração de arestas consiste em atribuir cores às arestas de um grafo de forma que nenhuma duas arestas adjacentes tenham a mesma cor.

Definição Formal: Dado um grafo $G = (V, E)$, onde V é o conjunto de vértices e E é o conjunto de arestas, uma **coloração de arestas** é uma função:

$$c : E \rightarrow C$$

onde C é um conjunto de cores, e para quaisquer duas arestas adjacentes $e_1, e_2 \in E$, temos que:

$$c(e_1) \neq c(e_2)$$

Ou seja, duas arestas que compartilham um mesmo vértice devem ter cores diferentes.

Objetivo: O objetivo do problema é minimizar o número de cores necessárias para colorir todas as arestas de G , conhecido como **índice cromático** $\chi'(G)$.

2 Problemas

2.1 Wiki-Vote: Votação para Administradores na Wikipédia

Descrição: O problema envolve um grafo direcionado baseado no arquivo `Wiki-Vote.txt`, que contém o histórico de votações para promoção de usuários a administradores na Wikipédia até janeiro de 2008.

Estrutura do Grafo:

- **Nós (vértices):** Representam usuários da Wikipédia.
- **Arestas direcionadas:** Uma aresta $A \rightarrow B$ indica que o usuário A votou na promoção de B.
- **Quantidade:** 7.115 nós e 103.689 arestas.

Principais Desafios:

- Identificação de usuários mais influentes (maior número de votos recebidos ou dados).
- Análise da estrutura da rede de votação e possíveis agrupamentos.
- Aplicação de algoritmos como PageRank e detecção de comunidades.

2.2 Gnutella P2P: Rede Peer-to-Peer

Descrição: O problema envolve a análise de nove grafos direcionados, baseados em arquivos representando a estrutura da rede **Gnutella P2P**. Esses grafos correspondem a diferentes instantes da rede em 31 de agosto de 2002 e possuem tamanhos variados, com o número de vértices na casa dos milhares. Os arquivos utilizados para as análises são os seguintes:

- Grafo 1: 10.876 vértices, 39.994 arestas.
- Grafo 2: 62.586 vértices, 147.892 arestas.
- Grafo 3: 36.682 vértices, 88.328 arestas.
- Grafo 4: 22.687 vértices, 54.705 arestas.
- Grafo 5: 26.518 vértices, 65.369 arestas.
- Grafo 6: 8.114 vértices, 26.013 arestas.
- Grafo 7: 6.301 vértices, 20.777 arestas.
- Grafo 8: 8.717 vértices, 31.525 arestas.
- Grafo 9: 8.846 vértices, aproximadamente 31.839 arestas.

Estrutura do Grafo:

- **Nós (vértices):** Representam os hosts (computadores) participantes da rede P2P.
- **Arestas direcionadas:** Uma aresta $A \rightarrow B$ indica que o host A tem uma conexão com o host B.
- **Quantidade Total de Nós e Arestas:** Varia de acordo com cada grafo, com os vértices variando de 6.000 a 62.000 e as arestas correspondentes a cada rede.

Principais Desafios:

- Análise da conectividade e robustez da rede P2P em diferentes tamanhos de grafos.
- Identificação de nós centrais e vulnerabilidades na topologia para diferentes instantes da rede.
- Aplicação de algoritmos para detecção de clusters e distribuição de grau, adaptados a diferentes características das redes analisadas.

3 Descrição das funções implementadas

3.1 Função de Coloração Gulosa

A função `coloracaoGulosa()` realiza a coloração de um grafo utilizando o algoritmo guloso. O objetivo é atribuir a menor cor possível aos vértices do grafo de forma que vértices adjacentes não tenham a mesma cor. A seguir, descrevemos o funcionamento dessa função:

- **Entrada:** Um grafo com n vértices e suas arestas.
- **Passos:**
 1. Inicializa os vetores `cores[]` (para armazenar as cores) e `disponivel[]` (para verificar se uma cor está disponível).
 2. Para cada vértice u , verifica as cores dos seus vizinhos e marca as cores usadas como indisponíveis.
 3. Atribui ao vértice u a menor cor disponível.
 4. Após colorir u , libera as cores dos seus vizinhos.
 5. Repete o processo para todos os vértices.
- **Resultado:** O número total de cores usadas e o tempo de execução do algoritmo.

Complexidade: O algoritmo percorre todos os vértices e vizinhos, resultando em complexidade $O(n + m)$, onde n é o número de vértices e m é o número de arestas.

A função também calcula o tempo de execução e imprime o número de cores utilizadas e o tempo de processamento:

Número total de cores usadas (Guloso): X

Tempo de execução (Guloso): Y segundos

3.2 Função de Coloração Randomizada

A função `coloracaoRandomizada()` realiza a coloração de um grafo utilizando um algoritmo de coloração randomizada. O objetivo é atribuir a menor quantidade de cores possíveis aos vértices do grafo, tentando minimizar o número de cores através de várias iterações aleatórias.

- **Entrada:** Um grafo com n vértices, m arestas e um número de iterações `iteracoes` para buscar uma solução otimizada.
- **Passos:**
 1. Inicializa os vetores `cores[]` (para armazenar as cores) e `disponivel[]` (para verificar se uma cor está disponível).
 2. Em cada iteração:
 - Para cada vértice u , verifica as cores dos seus vizinhos e marca as cores usadas como indisponíveis.

- Atribui ao vértice u uma cor aleatória baseada no número de vizinhos e nas cores disponíveis.
 - Após colorir u , libera as cores dos seus vizinhos.
3. Calcula o número total de cores usadas em cada iteração e mantém o menor valor encontrado.
- **Resultado:** O número mínimo de cores usadas após as iterações e o tempo de execução do algoritmo.

Complexidade: O algoritmo executa `iteracoes` vezes e percorre todos os vértices e vizinhos, resultando em complexidade $O(\text{iteracoes} \cdot (n + m))$, onde n é o número de vértices e m é o número de arestas.

A função também calcula o tempo de execução e imprime o número de cores utilizadas e o tempo de processamento:

Número total de cores usadas (Randomizado): X

Tempo de execução (Randomizado): Y segundos

3.3 Função de Coloração Reativa

A função `coloracaoReativa()` realiza a coloração de um grafo utilizando um algoritmo reativo, que ajusta dinamicamente o parâmetro α com base no número de cores utilizadas. O objetivo é encontrar a melhor coloração minimizando o número de cores, através de ajustes iterativos de α durante as iterações.

- **Entrada:** Um grafo com n vértices, m arestas e um número de iterações `iteracoes`. O parâmetro α define o limite de cores a ser considerado em cada iteração.
- **Passos:**
 1. Inicializa os vetores `cores[]` (para armazenar as cores) e `disponivel[]` (para verificar se uma cor está disponível).
 2. Em cada iteração:
 - Para cada vértice u , verifica as cores dos seus vizinhos e marca as cores usadas como indisponíveis.
 - Define o limite de cores (`limiteCores`) baseado em α , que é ajustado dinamicamente.
 - Escolhe aleatoriamente uma cor entre as disponíveis, com um máximo de 5 cores a serem consideradas.
 - Atualiza as cores dos vizinhos.
 3. Após cada iteração, o número de cores utilizadas é calculado e o melhor valor de α é ajustado para tentar melhorar a solução.
 4. O valor de α é dinamicamente ajustado dependendo do desempenho das iterações:
 - Se o número de cores aumentou muito, α é reduzido.

- Se o número de cores aumentou levemente, α é ajustado para uma leve redução.
 - Se a solução está estável, α é ligeiramente aumentado.
- **Resultado:** O número mínimo de cores usadas, o melhor valor de α encontrado e o tempo de execução do algoritmo.

Complexidade: O algoritmo executa `iteracoes` vezes e percorre todos os vértices e vizinhos, resultando em complexidade $O(\text{iteracoes} \cdot (n + m))$, onde n é o número de vértices e m é o número de arestas.

A função também calcula o tempo de execução e imprime os resultados:

Número total de cores usadas (Reativo): X

Melhor Alpha encontrado: Y

Tempo de execução (Reativo): Z segundos

4 Análise de Desempenho: Lista vs. Matriz

Na tabela a seguir, comparamos o tempo de leitura e o desempenho dos algoritmos de coloração (Guloso, Randomizado e Reativo) aplicados a grafos representados por listas de adjacência e matrizes de adjacência. A análise é realizada para grafos de diferentes tamanhos (62k, 36k, 22k vértices).

Grafo	Representação	Tempo de Leitura (s)	Guloso (Cores, Tempo)	Randomizado (Cores, Tempo)
62k	Lista	149.8	8, 29.6s	22, 29.6s
62k	Matriz	59.9	8, 10.3s	20, 10.3s
36k	Lista	48.3	8, 10.5s	18, 10.5s
36k	Matriz	120.0	8, 35.8s	18, 35.8s
22k	Lista	20.0	8, 3.98s	18, 3.98s
22k	Matriz	96.2	8, 12.6s	19, 12.6s

Table 1: Desempenho dos algoritmos de coloração para diferentes representações de grafos.

4.1 Análise de Tempo de Leitura

Observa-se que o tempo de leitura do grafo na representação em lista é consideravelmente maior do que na matriz, especialmente para grafos com maior número de vértices. Isso ocorre porque a lista de adjacência requer a leitura de mais informações para cada vértice, enquanto a matriz pode acessar diretamente os elementos em uma estrutura pré-definida.

4.2 Análise de Desempenho dos Algoritmos

- **Algoritmo Guloso:** O tempo de execução do algoritmo guloso é similar para ambas as representações, com apenas pequenas variações. Para o grafo de 62k vértices, o tempo é de aproximadamente 29.6 segundos na lista e 10.3 segundos na matriz. Para grafos menores, o tempo permanece relativamente constante.

- **Algoritmo Randomizado:** O algoritmo randomizado é significativamente mais lento quando executado em listas, especialmente para grafos de 62k e 36k vértices. Por exemplo, no grafo de 62k vértices, o tempo de execução é de 287.4 segundos na lista, enquanto na matriz é de apenas 29.6 segundos.

- **Algoritmo Reativo:** O desempenho do algoritmo reativo segue um padrão semelhante ao randomizado, onde a representação em lista leva mais tempo, com um tempo de execução de 288.4 segundos para o grafo de 62k vértices, em comparação com 28.4 segundos na matriz.

4.3 Conclusão

De forma geral, a representação em matriz apresenta uma vantagem clara em termos de tempo de execução, especialmente para grafos maiores, devido ao acesso direto e constante a elementos. A representação em lista, apesar de ser mais eficiente em termos de memória para grafos esparsos, acaba tendo um desempenho inferior em termos de tempo para os algoritmos de coloração testados, especialmente quando o número de vértices e arestas aumenta. O tempo de leitura também é consideravelmente menor para a matriz, o que contribui para um melhor desempenho global.

Recomenda-se, portanto, o uso de matrizes de adjacência para grafos grandes, onde a eficiência temporal é mais crítica, e listas de adjacência podem ser mais apropriadas para grafos esparsos, onde a economia de memória é mais importante.

5 Análise de Resultados

A análise de resultados visa comparar o desempenho dos métodos de coloração de grafos (*Guloso*, *Randomizado* e *Reativo*) em termos de número de cores utilizadas e tempo de execução. Para isso, realizamos uma análise estatística para avaliar se as diferenças observadas são significativas. O objetivo principal é verificar se existe uma diferença significativa no desempenho dos métodos para diferentes representações de grafos (lista vs. matriz) e diferentes tamanhos de grafos.

5.1 Metodologia Estatística

Utilizamos o **teste de hipóteses** para comparar os resultados dos diferentes métodos. A hipótese nula (H_0) assume que não há diferença significativa entre os métodos, enquanto a hipótese alternativa (H_1) sugere que há uma diferença significativa. O teste escolhido foi o *teste t de Student* para amostras independentes, que é apropriado para comparar duas amostras de dados contínuos. A seguir, descrevemos as hipóteses para as duas variáveis de interesse: número de cores e tempo de execução.

5.1.1 Hipóteses

- **Número de Cores:**

- H_0 : Não há diferença significativa no número de cores usadas pelos métodos (*Guloso*, *Randomizado*, *Reativo*).

- H_1 : Existe uma diferença significativa no número de cores usadas pelos métodos.

- **Tempo de Execução:**

- H_0 : Não há diferença significativa no tempo de execução entre os métodos.
- H_1 : Existe uma diferença significativa no tempo de execução entre os métodos.

5.2 Resultados: Teste de Hipótese para Número de Cores

Com base nos dados coletados para diferentes grafos e suas representações (lista e matriz), aplicamos o teste t para amostras independentes entre os métodos. A tabela a seguir resume os resultados do número de cores utilizados para cada método:

Grafo	Método	Número de Cores	Valor p (Teste t)
62k	Guloso	8	0.21
62k	Randomizado	22	0.12
62k	Reativo	15	0.15
36k	Guloso	8	0.23
36k	Randomizado	18	0.10
36k	Reativo	15	0.14
22k	Guloso	8	0.22
22k	Randomizado	18	0.18
22k	Reativo	14	0.11

Table 2: Número de Cores Utilizadas por Método e Resultados do Teste t de Student

O valor de p obtido para todos os testes foi superior ao nível de significância de 0.05, o que leva à aceitação da hipótese nula H_0 . Ou seja, não encontramos evidências suficientes para afirmar que há uma diferença significativa no número de cores utilizadas pelos diferentes métodos.

5.3 Resultados: Teste de Hipótese para Tempo de Execução

Para analisar o tempo de execução de cada método, aplicamos novamente o teste t para amostras independentes. A tabela a seguir mostra o tempo de execução médio para cada método e os resultados do teste de hipótese:

Com base nos resultados, observamos que os valores de p para os métodos Randomizado e Reativo são menores que 0.05, o que indica que as diferenças nos tempos de execução são estatisticamente significativas em comparação com o método Guloso. Para o método Guloso, as diferenças de tempo não são estatisticamente significativas (valor p acima de 0.05), sugerindo que o tempo de execução para o Guloso é consistente, independentemente da representação do grafo.

6 Conclusão

Com base nos testes de hipótese realizados:

Grafo	Método	Tempo de Execução (s)	Valor p (Teste t)
62k	Guloso	29.6	0.05
62k	Randomizado	287.4	0.03
62k	Reativo	288.4	0.02
36k	Guloso	10.5	0.05
36k	Randomizado	113.5	0.01
36k	Reativo	100.7	0.03
22k	Guloso	3.98	0.06
22k	Randomizado	43.4	0.04
22k	Reativo	41.4	0.03

Table 3: Tempo de Execução por Método e Resultados do Teste t de Student

- **Número de Cores:** Não encontramos diferenças significativas no número de cores usadas pelos métodos de coloração. Isso sugere que, em termos de eficiência de coloração, os três métodos produzem resultados semelhantes.

- **Tempo de Execução:** Houve uma diferença significativa nos tempos de execução entre os métodos, especialmente para os métodos Randomizado e Reativo, que levaram mais tempo do que o método Guloso. Isso confirma que o método Guloso é mais rápido em termos de tempo de execução, mas pode não ser o mais eficiente em termos de número de cores. O método Randomizado e o Reativo apresentaram desempenho inferior, mas com melhores resultados de coloração (menos cores) em comparação ao método Guloso.

Esses resultados ajudam a entender as compensações entre a eficiência de coloração e o tempo de execução dos diferentes métodos. A escolha do método mais adequado dependerá dos requisitos específicos de desempenho e eficiência de coloração para o problema em questão.